

Document number: P2601R0
Date: 2022-06-11
Reply-to: Justin Cooke <jgc@cems.de>
Target audience: EWG, CWG

Make redundant empty angle brackets optional

Proposal: To make redundant empty angle brackets optional for class template argument lists.

Motivation. Additional template parameters with default arguments can be added to a class template declaration in a header file without breaking code that uses the template. For example, early versions of `std::vector` took just one parameter, the value type; the allocator parameter was added later, with a default argument. However, that upgrade pathway is currently only open where there is already at least one template parameter. It is not currently possible to replace a non-template class declaration by a class template, even with default arguments for all parameters, without breaking most uses of the class, because the syntax for using class templates requires empty angle brackets in most circumstances, even where their omission would not result in ambiguity. Since the introduction of CTAD (in C++17), angle brackets can be omitted when defining objects, but they are still required when declaring pointers, references, arrays, external objects, function parameters, class members, base classes, or template type parameters. The following examples illustrate the current situation:

```
template <typename T = int> class C {};  
C c;                                //OK (1)  
extern C x;                          //error: need C<>  
C *p;                               //error: need C<>  
C a[2];                             //error: need C<>  
C &r = c;                           //error: need C<>  
struct S { C c; };                  //error: need C<>  
struct D : C {};                    //error: need C<>  
std::vector<C> v;                   //error: need C<>  
void foo(C c);                      //error: need C<>
```

The empty angle brackets are currently optional only in case (1). No ambiguity would arise from making them optional in all the above cases, because names of class or variable templates must be unique within a scope (13.1 Note 5). This change will allow class library maintainers to upgrade existing class definitions to templates without breaking user code.

Effect on existing code: None. Code using the proposed feature is ill-formed under C++20 and earlier standards.

Note: Empty angle brackets will still be required to disambiguate invocations of function templates from functions with the same name (13.10.2), and to denote empty template *parameter* lists, which are required to disambiguate explicit full specialization declarations (13.9.4) from explicit instantiation declarations (13.9.3).

Wording: (edits to N4910)

13.4.1 [temp.arg.general]

...

⁵ When template argument packs or default *template-arguments* are used, **or template arguments are deduced (13.10.3)** a *template-argument* list can be empty. In that case the empty `<>` brackets **may be omitted** ~~shall still be used as the template argument list~~ **unless needed to disambiguate an invocation**

of a function template specialization from an invocation of a non-template function with the same name (13.10.2).

[Example 5 :

```
template<class T = char> class String;  
String<>* p; //OK,String<char>  
String* q; // syntax error OK, empty brackets <> omitted  
template<class ... Elements> class Tuple;  
Tuple<>* t; //OK,Elements is empty  
Tuple* u; // syntax error OK, empty brackets <> omitted
```

—end example]

Two subsequent mentions (13.9.3 para 8, and 13.10.2 para 4) of the option to omit empty angle brackets become redundant under this proposal. If this redundancy grates, the repetitions can be deleted.